TrustNode Edge

Codebase Inventory & Customer-Branded Packaging Guide

Audit date: 2026-06-18 · Audit scope: complete repository

This report is a deep, read-only audit of the TrustNode codebase. It enumerates every directory and the role each plays, classifies files by sensitivity, and prescribes a maintainable recipe for shipping a **customer-branded Client View** deployment that points at the cloud TrustNode platform without exposing source code, secrets, or implementation internals to the customer.

Goal: let the customer (or their web team) host a self-contained, TrustNode-branded read-only portal on their own website, that reads *only* their scoped data from the TrustNode cloud, with no exposure of the desktop edge code, the backend Python services, the OPC/MQTT runtimes, or the control-plane.

Non-goal: distributing the desktop tray app, the .NET / Mosquitto sidecars, the FastAPI source, or any secret keys to the customer.

1. Executive summary

TrustNode ships in three layers: a Windows **desktop edge tray app** that runs inside the customer's plant, a **cloud control-plane + database** on the TrustNode VPS (Supabase Postgres), and a family of **web/lite views** that anyone with a browser can open. The packaging proposed here uses the same *web/lite* layer to create a per-customer, on-customer-domain, read-only portal that is operationally isolated from the proprietary edge and backend.

Three properties of the existing architecture make this clean:

- The **cloud Lite** at `/lite/index.html` is a single self-contained HTML file that loads React + Recharts from a public CDN (esm.sh) and Supabase from the public JS SDK. *No build step is required to deploy it.*
- Tenant isolation lives in the **database** via Supabase Row-Level Security, NOT in the client. A leaked Lite HTML cannot read another customer's data even if the anonymous key is exposed.
- A small `config.json` sits next to the HTML and carries all per-customer customisation (label, tenant scope, enabled tabs, polling cadence). Customisation is configuration, not source code.

Maintaining a customer-specific build therefore reduces to maintaining a small directory of static assets and a `config.json`. The underlying React bundle is reused across every customer; brand assets and configuration are the only forked surface.

> Recommended pattern: keep **one canonical Lite HTML** in this repo and produce per-customer folders that are *almost entirely brand assets + config*. Re-run a tiny copy script when the canonical Lite is rebuilt. No customer-side React build, no Vite, no Node.

2. Repository inventory

The full repo tree is reproduced below with each top-level folder's role. Sensitivity is rated relative to a customer-distribution risk model:

Rating	Meaning
Low	Public-safe. Brand assets, generic CSS, public CDN-based HTML, documented APIs.
Medium	Internal but not catastrophic if exposed. Migration SQL, frontend source, docs.
High	Must never reach the customer. Backend Python, edge tray app, secrets, .env, build scripts that

2.1 Top-level layout

Folder	Role
backend/	FastAPI edge service. Python app, runs inside the tray EXE AND on the cloud VPS.
frontend/	React desktop UI source. Vite build → dist/ for tray and cloud portal.
desktop/	Electron tray wrapper. main.js + NSIS installer hooks + Mosquitto/OPC sidecar staging.
web_cloud_readonly/	Public-facing static deploy roots. lite/, portal/, developer-portal/, brand PNGs.
db/migrations/	Supabase SQL migrations. Schema-of-record for the cloud database.
backend/sql/migrations/	Older migration set for the VPS-owned schema (control_plane_core, telemetry_v1).
docs/	Architecture whitepapers, scalability reports, deployment diagrams, decks.
scripts/	PowerShell/Python build + provisioning scripts (clientview build, customer provisioning, etc.).
tests/, backend/tests/	Smoke + unit tests. Never deployed.
client_page_tests/	Documentation + experimental shells for the single-file client variants.
backups/, .runlogs/, runlogs/, .runtime-logs/	Local-only operational logs. Excluded from any customer deploy.

2.2 Backend — backend/app/

Two sibling packages: `routers/` (HTTP endpoint groups) and `services/` (long-lived workers, persistence, gateway managers). Both are required for the FastAPI process and are bundled into the tray EXE via PyInstaller.

Routers

Path	Purpose	Sensitivity	Ship to customer
`backend/app/routers/auth.py`	Login + JWT issuance + temp passwords.	High	No
`backend/app/routers/control_plane.py`	Tenant/customer/edge admin, view-link mint, activation.	High	No
`backend/app/routers/plc.py`	Gateway runtime control, tag discovery, Modbus/AB/Snap7/OPC-UA browse.	High	No
`backend/app/routers/power.py`	Power-meter dashboard data + tariff math.	High	No
`backend/app/routers/reports.py`	PDF reporting, templates, schedules.	High	No
`backend/app/routers/notifications.py`	Email + SMTP test endpoints.	High	No
`backend/app/routers/app_store.py`	Local SQLite app-store get/put (config, historian, live, logs).	High	No
`backend/app/routers/database.py`	Local database management.	High	No
`backend/app/routers/historian_export.py`	XLSX export with formatting.	High	No
`backend/app/routers/connections.py`	OPC UA + MQTT toggle, runtime selector.	High	No

backend/app/routers/customer_db.py	Customer Postgres mode activation.	High	No
backend/app/routers/lan_sharing.py	LAN 0.0.0.0 toggle for in-plant Lite.	High	No
backend/app/routers/lite_local.py	View-link → session JWT exchange.	High	No
backend/app/routers/cloud_live.py	SSE stream of live data to cloud Lite.	High	No
backend/app/routers/telemetry_v1.py	Edge-side v1 ingest + query (device + user tokens).	High	No
backend/app/routers/ui_source.py	UI source config (local vs remote frontend).	Medium	No
backend/app/routers/health.py	Liveness probe.	Low	No

Services

Path	Purpose	Sensitivity	Ship to customer
backend/app/services/app_store.py	Local SQLite ORM-less layer (~7,300 LOC). Source of truth on the edge.	High	No
backend/app/services/plc_manager.py	Per-gateway worker. Allen-Bradley / Siemens / Modbus / OPC-UA loops.	High	No
backend/app/services/power_manager.py	Power-meter worker + tariff insights.	High	No
backend/app/services/report_renderer.py	PDF rendering via reportlab.	High	No
backend/app/services/report_scheduler.py	Cron-style timer + tag-trigger queue.	High	No
backend/app/services/reports_store.py	Report templates + history persistence.	High	No
backend/app/services/control_plane_store.py	SQLite-backed tenant/edge/license catalog.	High	No
backend/app/services/control_plane_supabase.py	Supabase-backed control plane (cloud variant).	High	No
backend/app/services/cp_users_puller.py	Pulls cp_users from cloud into local edge.	High	No
backend/app/services/customer_sql.py	Customer-Postgres engine pool + bootstrap.	High	No
backend/app/services/sinks_sql.py	Generic Postgres historian writer.	High	No
backend/app/services/ingest_store.py	Telemetry v1 ingest persistence.	High	No
backend/app/services/telemetry_service.py	Telemetry v1 service layer.	High	No
backend/app/services/lite_user_mirror.py	Mirrors local users into Supabase lite_profiles.	High	No
backend/app/services/lite_report_poller.py	Drains Supabase report queue on the cloud.	High	No
backend/app/services/reports_cloud_upload.py	Uploads PDFs to Supabase Storage.	High	No
backend/app/services/lan_socket.py	Second in-process uvicorn on 0.0.0.0.	High	No
backend/app/services/connections_publish.py	Dispatches tag updates to OPC/MQTT runtimes.	High	No
backend/app/services/opcua_server.py	asyncua-based Python OPC UA server.	High	No
backend/app/services/opcua_server_dotnet.py	Manager around the OPC Foundation .NET sidecar.	High	No
backend/app/services/mqtt_broker.py	amqtt-based Python MQTT broker.	High	No
backend/app/services/mqtt_broker_mosquitto.py	Manager around the Mosquitto sidecar + paho publisher.	High	No

2.3 Frontend — frontend/

Path	Purpose	Sensitivity	Ship to customer
frontend/src/App.jsx	The full React UI (~33,000 LOC). Same bundle in tray + cloud portal + cloud Lite.	Medium	Yes (as compiled output only)
frontend/src/api.js	Single source for /api/* + Supabase paths. Hits same endpoint normal users.	Medium	Yes
frontend/src/components/Dashboard/*	Dashboard editor + widgets + analytics.	Medium	No
frontend/src/components/Login/Login.jsx	Login surface + theming.	Medium	No
frontend/src/components/Icons/	SVG icon library.	Low	Yes if embedded by Lite
frontend/src/components/Reports/	Report builder UI.	Medium	No
frontend/src/styles*.css	Base + portal + client + local theme sheets.	Low	Yes (when Lite uses them)
frontend/vite.config.js	Build modes: default, cloudro, clientview.	Medium	No
frontend/.env.clientview	Sets VITE_TRUSTNODE_CLIENT_VIEW=true. No secret here.	Low	No
frontend/.env.cloudro	Sets VITE_TRUSTNODE_READONLY=true. No secret here.	Low	No

`frontend/dist/`	Build output. Ships into tray EXE + /var/www/trustnode/.	Low	Indirect (via cloud)
`frontend/public/`	Static brand PNGs copied to dist.	Low	Yes

2.4 Desktop — desktop/

Path	Purpose	Sensitivity	Ship to customer
`desktop/main.js`	Electron tray entry. Spawns backend, manages LAN/firewall, sidecar lifecycle.	High	No
`desktop/preload.js`	IPC bridge for renderer.	Medium	No
`desktop/package.json`	Electron + electron-builder config. NSIS hooks. Per-machine install.	Medium	No
`desktop/build/installer.nsh`	Windows Firewall rule adds at install time.	Low	No
`desktop/assets/`	Tray icon, brand PNG.	Low	Yes (brand only)
`desktop/dist/`	Built NSIS + portable EXEs (~165 MB each).	High	No

2.5 Cloud static — web_cloud_readonly/

This is the only folder that public users reach directly. Everything here is intentionally distributable, with the caveat that `lite/config.json` may carry per-deployment branding and the Supabase anon key.

Path	Purpose	Sensitivity	Ship to customer
`web_cloud_readonly/index.html`	Brand redirect into / (the React app).	Low	Optional
`web_cloud_readonly/lite/index.html`	Single-file React Lite via CDN imports (~4,600 LOC, 200 KB).	Low	Yes (canonical Lite)
`web_cloud_readonly/lite/config.json`	Per-deployment Supabase URL + anon key + tab toggles + branding label.	Medium	Yes (customised)
`web_cloud_readonly/lite/config.json.template`	Template for a fresh deployment.	Low	Yes
`web_cloud_readonly/lite/styles.css`	Lite-specific styling.	Low	Yes
`web_cloud_readonly/lite/manifest.json`	PWA manifest.	Low	Yes
`web_cloud_readonly/lite/sw.js`	Service worker (optional offline cache).	Low	Yes
`web_cloud_readonly/lite/trustnode_app_icon*.png`	App icons (PWA + apple-touch).	Low	Yes
`web_cloud_readonly/lite/trustnode_login_logo.png`	Brand logo on login.	Low	Yes
`web_cloud_readonly/lite/trustnode_logo.png`	Brand logo in header.	Low	Yes
`web_cloud_readonly/developer-portal/index.html`	Developer-only redirect to /developer-portal/.	Low	No
`web_cloud_readonly/portal/v1/index.html`	Builder-loading skeleton; portal v1 deploys.	Low	No
`web_cloud_readonly/assets/`	Hashed JS/CSS used by /portal/.	Medium	No
`web_cloud_readonly/trustenode-004.png`	Public brand asset.	Low	Yes

2.6 Database migrations — db/migrations/

Path	Purpose	Sensitivity	Ship to customer
`db/migrations/20260517_lite_realtime.sql`	Realtime publication for live_latest + dashboards.	Medium	No
`db/migrations/20260517_lite_rls.sql`	Row-Level Security policies on all data tables.	Medium	No
`db/migrations/20260517_dashboard_configurations.sql`	Dashboard configurations table + audit log.	Medium	No
`db/migrations/20260518_per_customer_tenancy.sql`	Per-customer tenancy enforcement.	Medium	No
`db/migrations/20260518_*.sql`	Reports queue, historian indexes, etc.	Medium	No
`db/migrations/20260519_align_cp_schema_to_cloud.sql`	Schema alignment for cloud store.	Medium	No
`db/migrations/20260610_gateway_device_mirror.sql`	Edge-to-cloud device mirror.	Medium	No
`db/migrations/20260611_lite_profile_auto_create.sql`	Auto-create lite_profiles on first login.	Medium	No
`db/migrations/20260611_historian_tail_index.sql`	Tail-query index for historian.	Medium	No
`db/migrations/20260617_view_links_per_user.sql`	Per-user view-link tokens (user_id column).	Medium	No

3. Data flow and what the customer actually sees

Five layers, three roles, one canonical bundle. The customer-distributable surface is the green band only.

3.1 Layers

Layer	Where it lives + who owns it
1. PLC / meter	Customer's plant network. TrustNode reads via gateway protocols.
2. Edge tray (Windows)	Customer-installed EXE. SQLite locally; pushes to cloud.
3. Cloud control-plane + DB	TrustNode VPS + Supabase. trustnode.lsapps.app.
4. Cloud Lite + Portal	/lite/index.html (read-only Supabase) + / (full React portal).
5. Customer-hosted Client View	(the new layer) A copy of layer 4 on the customer's domain.

3.2 Tenant scoping

TrustNode is multi-tenant. Every row in the cloud database carries a `tenant_id`. Each customer has their own tenant (see `20260518_per_customer_tenant.sql`). Row-Level Security policies (`20260517_lite_rls.sql`) enforce tenant separation at the database — even if a customer obtains another customer's anonymous key, Supabase will return zero rows because their user JWT does not satisfy the policy.

Key consequence: a customer Client View hosted on the customer's own domain is just a static HTML + config.json. It cannot impersonate another tenant. The trust boundary is the Supabase JWT and the lite_profiles row, not the page.

3.3 What is hidden from the customer

- All Python code (`backend/`).
- All React source (`frontend/src/`). The customer receives only the compiled, minified bundle hosted on the cloud.
- Desktop tray (`desktop/`) — only the operator installs it on plant PCs.
- Build scripts that embed VPS credentials (`scripts/_*`, .env files).
- Migrations + smoke tests — internal engineering only.
- Whitepapers, decks, internal docs in `docs/`.
- Native sidecars (`backend/sidecars/`) and the C# sidecar source (`backend/sidecars/opcua-cs/`).

3.4 What is exposed to the customer (the green band)

- One HTML file (the cloud Lite).
- One `config.json` with their tenant_id, customer label, tab toggles, polling intervals.
- Brand PNGs (logo, login background, app icons).
- CSS + service worker + PWA manifest.
- Public Supabase URL + *anon* public key. Security comes from RLS, not key secrecy.

4. The customer folder — recommended layout

Below is a maintainable folder layout intended to be checked into a small per-customer git repository (or shipped to the customer's web team as a zip). It is small enough to audit by eye, contains zero proprietary code, and is updated by re-running a single synchronisation script when the canonical Lite is rebuilt.

```
trustnode-customer-<NAME>/
■■■ README.md ← how to update + how to host
■■■ public/ ← drop-in for any static web host
■ ■■■ index.html ← copy of /lite/index.html
■ ■■■ styles.css ← copy of /lite/styles.css
■ ■■■ manifest.json ← copy of /lite/manifest.json
■ ■■■ sw.js ← copy of /lite/sw.js
■ ■■■ config.json ← CUSTOMER-SPECIFIC
■ ■■■ trustnode_app_icon.png
■ ■■■ trustnode_app_icon_180.png
■ ■■■ trustnode_login_logo.png ← customer brand override
■ ■■■ trustnode_logo.png ← customer brand override
■■■ source/ ← upstream reference (do not edit)
■ ■■■ lite_index.html.upstream.txt ← hash of last sync
■ ■■■ lite_styles.css.upstream.txt
■■■ sync.ps1 ← copies the canonical Lite into public/
```

Operational philosophy: the `public/` folder is what the customer hosts. `config.json` and the brand PNGs are the ONLY files the integrator edits per customer. Everything else comes from the canonical Lite via `sync.ps1`.

4.1 config.json fields and how to fill them

Field	Value + notes
supabase_url	https://<PROJECT-REF>.supabase.co · same for all customers · safe to commit
supabase_anon_key	Public anon key from the Supabase project · safe to commit · security comes from RLS
customer_label	Shown in the Lite header. e.g. "Acme Bottling Plant".
tenant_id	The customer's tenant in cp_customers. e.g. tenant-acme. Informational only.
show_dashboard_tab	true / false
show_alarms_tab	true / false
show_power_tab	true / false
live_poll_ms	Default 2000. Lower = livelier, more bandwidth.
history_default_limit	Default 300 rows.
default_chart_range_minutes	Default 60.

4.2 sync.ps1 — the recommended sync recipe

The script keeps the canonical cloud Lite as the source of truth. When the platform team rebuilds the Lite (because of a new widget type, a chart fix, a brand refresh), the integrator runs `sync.ps1` on every customer folder to pull in the latest. Brand assets and `config.json` are left untouched by the sync.

```
param([string]$CanonicalRoot = "<path-to-Trustnode_edge_app>\web_cloud_readonly\lite")
$dest = Join-Path $PSScriptRoot "public"
```

```
$keep = @("config.json", "trustnode_login_logo.png", "trustnode_logo.png")
Get-ChildItem $CanonicalRoot -File | ForEach-Object {
  if ($keep -contains $_.Name) { return }
  Copy-Item $_.FullName -Destination (Join-Path $dest $_.Name) -Force
}
Write-Host "Sync complete. Review public/config.json before deploy."
```

4.3 Hosting

- **Static host** (Netlify, Vercel, Cloudflare Pages, customer's own nginx): upload `public/`. No build step.
- **Customer's website** (e.g. customer.com/portal/): drop the contents of `public/` into `/portal/` under their document root. Adjust the manifest's `scope` + `start_url` if not at the host root.
- **CDN**: upload to S3 / R2 / GCS bucket with public read. Configure CORS only if the canonical Supabase URL refuses requests from the new origin (it normally accepts any).

4.4 Brand customisation

Replace the two PNGs (`trustnode_login_logo.png`, `trustnode_logo.png`) with co-branded artwork. Optionally replace the app icon PNGs to change the install-to-home-screen badge. Update `customer_label` in `config.json`. Avoid editing `index.html` directly — any change is overwritten by the next `sync.ps1`.

If the customer requires a colour palette override that isn't in config.json today, the cleanest path is to add a few CSS variables to `public/brand.css` and include it after `styles.css`. Long-term, those variables should be promoted into `config.json` so customisation stays declarative.

5. Sensitive surface + secrets to never ship

Treat the customer folder as if it will be publicly inspected. Everything below must stay behind the platform boundary.

Asset	Where it lives + why it must not ship
.env / .env files	Cloud DB password, Supabase service key, SMTP creds, VPS root password. NEVER commit a
TRUSTNODE_SUPABASE_SERVICE_KEY	Bypasses RLS. Whoever has this can read every customer's data.
VPS_HOST / VPS_USER / VPS_PASSWORD	Shell access to the production VPS.
TRUSTNODE_CLOUD_DB_PASSWORD	Direct Postgres write access.
backend/*.py	Tenant logic, control-plane logic, billing/license logic.
frontend/src/*.jsx	Trade-secret in the layout/widget editor + designer.
scripts/_*.py + apply-*.ps1	Operational scripts that embed cloud creds at runtime.
backend/sidecars/opcua-cs/	C# source for the OPC UA sidecar. Compiled binary is OK to ship inside the EXE; source is not.
docs/TrustNode_Security_*	Internal security whitepapers.
desktop/dist/	Built EXEs. Customer-facing only via the official download URL.

5.1 Threat model for the customer Client View

Threat	Mitigation
Customer modifies the HTML to query another tenant	Supabase RLS rejects the query — no rows returned.
Anonymous key leaked	Public by design. Same outcome — RLS denies cross-tenant reads.
Customer hosts the HTML on a malicious URL	Lite JWT still required; no cross-origin credential leak as no cookies are used.
Customer attempts to write data	Lite is read-only; write paths are not implemented; RLS denies writes from anon.
Customer attempts to call backend /api/* directly	Backend rejects without a valid Bearer JWT (cp_users authoritative).
Customer extracts edge source from the EXE	PyInstaller bytecode is not protected from reverse engineering. Defence: avoid shipping the EXE

5.2 Secret rotation hooks

If a customer-hosted Client View is ever suspected compromised, the response is to **rotate the customer's user passwords** (or revoke their per-user view-link tokens via Control Plane → Users → Lite Access → Rotate). Neither action requires touching the Lite file. The anon key itself does not need rotation because RLS makes it impotent.

Service-role keys and DB passwords are unrelated to Lite and should be rotated only on their own schedule (e.g., annually or on staff change).

6. Maintenance + release flow

Three release tracks. Customer folders depend on track 2 only.

Track	Cadence + what changes
1. Edge tray EXE	On-demand. desktop/dist/ rebuilt + signed + shipped to operators. Customer folders unaffected.
2. Canonical Lite (web_cloud_readonly/lite/)	When a customer-visible bug/feature lands. Re-run sync.ps1 in every customer folder.
3. Cloud control-plane (backend on VPS)	On commit to main + restart. Customer folders pick up the new data shapes automatically becau

6.1 Customer-folder update checklist

- Pull the latest `main` in the TrustNode repo.
- In each `trustnode-customer-<NAME>/` repo, run `.\sync.ps1`.
- Inspect `public/config.json` — it must be untouched.
- Verify brand PNGs still present.
- Commit + push the customer repo.
- If hosted on Netlify/Vercel, deploy auto-fires from the push.
- Smoke: open the URL, log in with one cloud user, see expected dashboards.

6.2 Schema breaks

When a Supabase migration changes a Lite-visible column or table, both the canonical Lite and any customer folder must be re-synced. The current architecture loads React + Recharts from esm.sh, so a bumped column rarely requires a Lite change — only a chart rename or a removed table does. Apply migrations to Supabase before customers reload their Lite to avoid a brief schema mismatch.

6.3 Adding a new customer

- In cloud Control Plane → Customers → New: create the customer, get its tenant_id.
- Create the customer's admin user (Control Plane → Users → New).
- Clone the customer-folder template: `cp -r trustnode-customer-template trustnode-customer-acme`.
- Edit `public/config.json`: tenant_id + customer_label.
- Drop in customer-branded PNGs (optional).
- Run `.\sync.ps1` to pull the current canonical Lite.
- Host on the customer's chosen origin.

7. Open considerations + recommendations

7.1 Lite versioning

The cloud Lite currently has a CSS cache-buster (`?v=20260612-1700`) but no version metadata in the HTML itself. Recommend adding a `<meta name="tn-lite-version">` tag and printing the build hash in the footer, so an integrator can verify which build a given customer is running.

7.2 Brand-variable promotion

Today, palette overrides need a side-file `brand.css`. Recommend promoting brand colours (--brand, --teal, --bg, --card) into config.json so customer customisation stays JSON-only and a single integrator can configure dozens of tenants without touching CSS.

7.3 Multi-domain Supabase Auth

When a customer hosts Lite on their own domain, Supabase Auth still works (no cookies, only the user JWT in localStorage). Confirm the redirect URLs in Supabase Auth settings include the customer's domain if magic-link email login is enabled.

7.4 Operational hygiene

- Keep customer folders in a *separate* git org so accidental pushes never leak into the main TrustNode repo.
- Never share VPS credentials with the customer or their integrator.
- Disable Supabase service-role key usage from any browser-facing code.
- Log every customer-folder sync to a CHANGELOG.md inside the customer repo for traceability.

Appendix A — what ships where

Audience	Receives
Operator (plant PC)	TrustNode-Setup.exe (installer EXE) + activation code
Cloud team	Full repository (git clone)
Customer's web team	trustnode-customer-<NAME>/ folder only (public/ + sync.ps1 + README)
End user (browser)	The hosted Lite URL. No downloads. No installation.
TrustNode internal	docs/, scripts/, .env files, decks

Appendix B — minimum customer folder file list

Smallest viable folder for a working customer-branded portal. Total size: about 700 KB.

File	Origin
public/index.html	Copy of web_cloud_readonly/lite/index.html
public/styles.css	Copy of web_cloud_readonly/lite/styles.css
public/manifest.json	Copy of web_cloud_readonly/lite/manifest.json
public/sw.js	Copy of web_cloud_readonly/lite/sw.js
public/config.json	From web_cloud_readonly/lite/config.json.example, filled in for the customer
public/trustnode_app_icon.png	Customer brand (or canonical fallback)
public/trustnode_app_icon_180.png	Customer brand (or canonical fallback)
public/trustnode_login_logo.png	Customer brand
public/trustnode_logo.png	Customer brand
README.md	Operator-written
sync.ps1	Operator-written (see § 4.2)

Appendix C — what NEVER goes in a customer folder

Item	Reason
.env / .env.* / .env.example	Carries secrets even in 'example' form (project ref)
backend/	Trade secret, license logic, control-plane logic
frontend/src/	Source for the React app — only the compiled cloud bundle is publicly served
desktop/	Tray app implementation
backend/sidecars/opcua-cs/	C# source for OPC UA sidecar
scripts/	Operational scripts
db/migrations/	Internal schema design
docs/	Decks + whitepapers + security architecture details
tests/, backend/tests/	Internal test infrastructure
backups/, *.log	Audit + scratch data

Report compiled by TrustNode engineering. For questions about packaging a specific customer, open an internal ticket referencing this report and quote the customer's tenant_id.